

CS 224 Implementation Details

RV32M Extension + Custom Mathematical Acceleration Unit

Verilog Pseudocode | Algorithms | Test Vectors | Pipeline Integration

[Team Name / Members]

March 2026

1. Complete Instruction Encoding Reference

This section provides the complete binary encoding for all 13 new instructions being added to the processor. The RV32M instructions use the standard OP opcode (0110011) with funct7 = 0000001, while custom MAU instructions use the custom-0 opcode (0001011) with funct7 = 0000001. All instructions follow the R-type format: [funct7(7) | rs2(5) | rs1(5) | funct3(3) | rd(5) | opcode(7)].

Instruction	Opcode (6:0)	funct3 (14:12)	funct7 (31:25)	Mnemonic
MUL	0110011 (OP)	000	0000001	rd = rs1[31:0] * rs2[31:0]
MULH	0110011 (OP)	001	0000001	rd = (rs1 * rs2)[63:32], signed
MULHSU	0110011 (OP)	010	0000001	rd = (signed rs1 * unsigned rs2)[63:32]
MULHU	0110011 (OP)	011	0000001	rd = (rs1 * rs2)[63:32], unsigned
DIV	0110011 (OP)	100	0000001	rd = rs1 / rs2, signed
DIVU	0110011 (OP)	101	0000001	rd = rs1 / rs2, unsigned
REM	0110011 (OP)	110	0000001	rd = rs1 % rs2, signed
REMU	0110011 (OP)	111	0000001	rd = rs1 % rs2, unsigned
ABS	0001011 (CUSTOM-0)	000	0000001	rd = rs1
MAX	0001011 (CUSTOM-0)	001	0000001	rd = max(rs1, rs2), signed
MIN	0001011 (CUSTOM-0)	010	0000001	rd = min(rs1, rs2), signed
SQRT	0001011 (CUSTOM-0)	011	0000001	rd = isqrt(rs1), unsigned
LOG2	0001011 (CUSTOM-0)	100	0000001	rd = floor(log2(rs1)), 0 if rs1=0

Note: For LOG2(0), we return 0 (not -1) to keep the result as a valid unsigned 32-bit value. Alternatively, your C test code can handle the zero-input case as a special branch. Both conventions are acceptable as long as they are consistent.

2. Multiplier Module (multiplier.v)

The multiplier performs 32x32-bit multiplication, producing a 64-bit result. All four variants (MUL, MULH, MULHU, MULHSU) are computed simultaneously, and the appropriate 32-bit slice is selected based on the funct3 control signal. The implementation uses a 3-stage internal pipeline with DSP slices on the FPGA.

2.1 Module Interface

```
module multiplier (  
    input wire    [31:0] operand_a,    // rs1 value  
    input wire    [31:0] operand_b,    // rs2 value  
    input wire      start,              // start computation  
    input wire      clk,
```

```

input wire          rst,
output reg [63:0] product, // full 64-bit result
output reg          valid   // result ready
);

```

2.2 Algorithm Description

The multiplier sign-extends both operands to 64 bits, then performs a 64x64-bit multiplication. For signed x signed (MULH), both operands are sign-extended. For unsigned x unsigned (MULHU), both are zero-extended. For signed x unsigned (MULHSU), rs1 is sign-extended and rs2 is zero-extended. The 64-bit result is always available at the product output, and the execute stage selects the appropriate 32-bit slice based on the funct3 field.

2.3 Result Selection Logic

```

// In execute.v - select result based on funct3
always @(*) begin
    case (funct3)
        3'b000: mul_result = product[31:0]; // MUL
        3'b001: mul_result = product[63:32]; // MULH (s*s)
        3'b010: mul_result = product_mulsu[63:32]; // MULHSU (s*u)
        3'b011: mul_result = product[63:32]; // MULHU (u*u)
        default: mul_result = 32'b0;
    endcase
end

```

2.4 Key Edge Cases for Testbench

The following edge cases must be explicitly tested in the multiplier testbench to ensure correct behavior across all four multiplication variants:

MUL: $0*0=0$, $1*1=1$, $(-1)*(-1)=1$, INT_MAX*INT_MAX (overflow), INT_MIN*INT_MIN (overflow), random pairs.

MULH: $(-1)*1$ = upper 32 bits of $0xFFFFFFFFFFFFFFFF = -1$, INT_MIN*1 upper bits = -1.

MULHU: UINT_MAX*UINT_MAX upper bits = $0xFFFFFFF$, $1*UINT_MAX$ upper bits = 0.

MULHSU: $(-1)*1$ upper bits = -1, INT_MIN*UINT_MAX upper bits = check RISC-V spec.

Note: When testing MULH/MULHSU, the golden output is the upper 32 bits of the 64-bit product. Use Python: `(a_signed * b_signed) >> 32`, masking to 32 bits for unsigned comparison.

3. Divider Module (divider.v)

The divider implements a restoring (or non-restoring) division algorithm for unsigned operands, with a sign-handling wrapper for signed DIV and REM. The divider uses a 32-step FSM, taking approximately 32-34 clock cycles per division operation.

3.1 Module Interface

```

module divider (
    input wire [31:0] dividend,

```

```

input wire      [31:0] divisor,
input wire      start,
input wire      is_signed, // 1 for DIV/REM, 0 for DIVU/REMU
input wire      clk,
input wire      rst,
output reg [31:0] quotient,
output reg [31:0] remainder,
output reg      done
);

```

3.2 Algorithm: Restoring Division

The restoring division algorithm operates as follows. A 64-bit remainder register (R) is initialized with the dividend in the upper 32 bits. In each of 32 iterations, the remainder is shifted left by 1, the divisor is subtracted from the upper 32 bits of the remainder, and if the result is non-negative, the quotient bit is set to 1 (the subtraction result is kept); otherwise, the quotient bit is 0 and the remainder is restored by adding the divisor back. After 32 iterations, the quotient is in the lower 32 bits of the quotient register and the remainder is in the upper 32 bits.

3.3 FSM States

```

typedef enum logic [1:0] {
    IDLE = 2'b00, // Waiting for start signal
    COMPUTE = 2'b01, // 32-iteration division loop
    SIGN_ADJ = 2'b10, // Adjust sign for signed division
    DONE = 2'b11 // Assert done, output valid
} state_t;

```

3.4 Signed Division Edge Cases (Critical)

The RISC-V specification mandates the following behavior for signed division edge cases. These are the most common source of bugs and must be handled explicitly:

Edge Case	DIV (signed)	REM (signed)	DIVU	REMU
Divisor = 0	Quotient = -1, Remainder = Dividend	Remainder = Dividend	Quotient = $2^{32}-1$, Remainder = Dividend	Remainder = Dividend
INT_MIN / -1	Quotient = INT_MIN (overflow)	Remainder = 0	N/A (unsigned)	N/A
Dividend = 0	Quotient = 0, Remainder = 0	Remainder = 0	Quotient = 0, Remainder = 0	Remainder = 0
1 / -1	Quotient = -1, Remainder = 0	Remainder = 0	N/A	N/A

Note: INT_MIN = 0x80000000. The division INT_MIN / -1 overflows because the mathematically correct result (2^{31}) cannot be represented as a signed 32-bit integer. The RISC-V spec requires returning INT_MIN (0x80000000) as the quotient and 0 as the remainder.

4. MAU Simple Operations (mau_simple.v)

The simple operations (ABS, MAX, MIN) are fully combinational and complete in a single clock cycle. They are implemented as simple logic circuits within the MAU module.

4.1 ABS - Absolute Value

```
// ABS: if rs1[31] is set (negative), negate; else pass through
assign abs_result = (operand_a[31]) ? (~operand_a + 1) : operand_a;

// Edge case: ABS(INT_MIN) = INT_MIN (0x80000000)
// This is automatically correct because ~INT_MIN + 1 = INT_MIN
// (two's complement overflow wraps around to same value)
```

The absolute value operation uses the two's complement identity: negating a number is equivalent to inverting all bits and adding 1. The sign bit (bit 31) of the input determines whether negation is needed. A critical edge case is $\text{ABS}(\text{INT_MIN}) = \text{INT_MIN}$, which is mathematically incorrect (the true absolute value is 2^{31} , which cannot be represented in 32 bits) but is the standard behavior for two's complement arithmetic and matches most programming language conventions.

4.2 MAX - Maximum of Two Signed Values

```
// MAX: compare rs1 and rs2 as signed values
wire [32:0] diff = {1'b0, operand_a} - {1'b0, operand_b};
// If diff is positive (no borrow), operand_a >= operand_b
assign max_result = (~diff[32]) ? operand_a : operand_b;

// Alternatively, using the sign bit of subtraction:
// Note: subtraction overflow can give wrong sign, so use 33-bit arithmetic
```

The MAX operation computes the difference between the two operands using 33-bit arithmetic (an extra bit to detect overflow in signed subtraction). If the result is non-negative (bit 32 is 0), operand_a is the maximum; otherwise, operand_b is the maximum. Using 33-bit arithmetic avoids the overflow bug where subtracting a large negative number from a large positive number could incorrectly indicate the wrong operand is larger.

4.3 MIN - Minimum of Two Signed Values

```
// MIN: inverse of MAX
wire [32:0] diff = {1'b0, operand_a} - {1'b0, operand_b};
assign min_result = (~diff[32]) ? operand_b : operand_a;
```

5. CLZ Unit (clz_unit.v)

The Count Leading Zeros (CLZ) unit is the hardware foundation for the LOG2 instruction. It counts the number of consecutive zero bits starting from the most significant bit (bit 31) of a 32-bit input. The LOG2 result is then computed as $31 - \text{CLZ}(\text{input})$ for non-zero inputs.

5.1 Algorithm: Hierarchical Priority Encoder

```
// 5-level tree: [32] -> [16] -> [8] -> [4] -> [2] -> [1]
function automatic [4:0] clz_32;
    input [31:0] val;
    begin
        if (val[31:16] != 0)
```

```

        clz_32 = clz_16(val[31:16]);
    else
        clz_32 = 5'd16 + clz_16(val[15:0]);
    end
endfunction

// clz_16: checks bits [15:8], adds 8 if upper half is zero
// clz_8:  checks bits [7:4],  adds 4 if upper half is zero
// clz_4:  checks bits [3:2],  adds 2 if upper half is zero
// clz_2:  checks bit  [1],    adds 1 if bit 1 is zero
// Base case: returns 0 if non-zero, 1 if zero

```

The hierarchical approach reduces the CLZ computation to $O(\log n)$ levels of multiplexer logic, yielding a combinational delay of approximately 5 LUT-levels on a Xilinx FPGA. This is significantly faster than the iterative approach (which would require 32 clock cycles) and maps efficiently to the FPGA's lookup table architecture.

5.2 LOG2 Derivation

```

// LOG2: floor(log2(x)) for x > 0
// Special case: x = 0, return 0 (or -1, depending on convention)
wire [4:0] zeros = clz_32(operand_a);
wire [31:0] log2_result = (operand_a == 0) ? 0 : (31 - zeros);

// Examples:
// log2(1) = log2(000...0001) = 31 - 31 = 0
// log2(2) = log2(000...0010) = 31 - 30 = 1
// log2(8) = log2(000...1000) = 31 - 28 = 3
// log2(255) = 31 - 24 = 7
// log2(256) = 31 - 23 = 8

```

6. Integer Square Root Unit (sqrt_unit.v)

The integer square root unit computes the floor of the square root of a 32-bit unsigned integer. It uses the non-restoring algorithm, which avoids the conditional addition step of the restoring algorithm, resulting in a more regular hardware structure.

6.1 Non-Restoring Square Root Algorithm

The non-restoring algorithm operates on the observation that the square root can be computed bit-by-bit, from the most significant to the least significant. The algorithm maintains a partial remainder R and a partial root Q , both initialized to 0. In each of 16 iterations (for a 32-bit input producing a 16-bit result), the following steps are performed: the partial remainder is shifted left by 2, the next bit from the input is appended, and a trial subtraction of $(2Q + 1)$ is performed. If the partial remainder is non-negative, the root bit is set to 1 and the subtraction result is kept. If negative, the root bit is 0 and the partial remainder is restored by adding $(2Q + 1)$ instead.

6.2 Module Interface

```

module sqrt_unit (
    input wire    [31:0] radicand, // input value
    input wire    start,

```

```

input wire      clk, rst,
output reg [15:0] root,    // integer square root
output reg      done
);

```

6.3 FSM Description

```

typedef enum logic [1:0] {
    S_IDLE   = 2'b00, // Wait for start
    S_COMPUTE = 2'b01, // 16 iterations
    S_DONE    = 2'b10  // Output result
} sqrt_state_t;

// Registers
reg [31:0] rem;    // partial remainder
reg [15:0] q;      // partial root
reg [4:0]  count;  // iteration counter (0 to 15)
reg [1:0]  state;

```

6.4 Verification Examples

The following test cases verify the SQRT unit across the input range:

Input (radicand)	Expected root	Verify: root * root <= input
0	0	0 * 0 = 0 <= 0
1	1	1 * 1 = 1 <= 1
4	2	2 * 2 = 4 <= 4
15	3	3 * 3 = 9 <= 15, 4*4=16 > 15
256	16	16 * 16 = 256 <= 256
1000	31	31 * 31 = 961 <= 1000
65535	255	255 * 255 = 65025 <= 65535
65536	256	256 * 256 = 65536 <= 65536
0xFFFFFFFF	65535	65535 * 65535 = 4294836225

7. Pipeline Integration

Integrating the new instructions into the 3-stage pipeline requires modifications to several existing modules. The key challenge is handling multi-cycle operations (MUL, DIV, SQRT) without violating pipeline correctness. Single-cycle operations (ABS, MAX, MIN, LOG2) are straightforward to integrate since they do not require stall logic.

7.1 Files to Modify

File	Changes Required
opcode.vh	Add OP_M (0110011) decode for RV32M. Add CUSTOM_0 (0001011) decode for MAU. Define new control signals: mul_en, div_en, mau_en, is_signed, mau_funct3.

execute.v	Route RV32M opcodes to multiplier/divider. Route MAU opcodes to mau_simple/sqrt/clz units. Add result multiplexer for all new units. Generate stall signal for multi-cycle ops.
IF_ID.v	Ensure stall from execute stage properly freezes the IF/ID pipeline register (insert bubble on stall).
memory.v	Minor: pass through any new control signals that affect write-back.
wb.v	Route write-back data from multiplier/divider/MAU result to register file. Ensure write-enable is asserted only when valid result is ready.
pipeline.v	Wire stall signal from execute to IF/ID stage. Ensure PC does not advance during multi-cycle operation.
top_fpga.v	No changes needed unless adding UART/7-seg peripherals.
constraint.xdc	Add timing constraints for new modules if needed. Pin assignments for new peripherals.

7.2 Stall Logic for Multi-Cycle Operations

```
// In execute.v: stall signal generation
wire mul_stall = mul_en & ~mul_valid;    // waiting for multiplier
wire div_stall = div_en & ~div_done;     // waiting for divider
wire sqrt_stall = sqrt_en & ~sqrt_done;  // waiting for sqrt
wire execute_stall = mul_stall | div_stall | sqrt_stall;

// This stall signal goes to:
// 1. IF stage: prevent PC from incrementing
// 2. IF_ID register: freeze (do not update with new instruction)
// 3. Control: prevent new instructions from entering execute stage
```

7.3 Result Write-Back Logic

```
// In wb.v: write-back data selection
always @(*) begin
    case (alu_src)
        ALU_NORMAL: wb_data = alu_result;    // existing
        ALU_MUL:    wb_data = mul_result[31:0]; // from multiplier
        ALU_DIV:    wb_data = div_result;     // from divider
        ALU_MAU:    wb_data = mau_result;     // from MAU
        default:    wb_data = alu_result;
    endcase
end

// Write enable: only when result is valid
wire wb_en = (reg_write & ~execute_stall) |
    (mul_en & mul_valid) |
    (div_en & div_done) |
    (mau_en & mau_done);
```

7.4 Timing Diagram: Multi-Cycle MUL

The following timing diagram illustrates pipeline behavior during a MUL instruction. The execute stage asserts the stall signal for 2 additional cycles (total 3 cycles for MUL), freezing the IF and ID stages while the multiplier completes its computation:

Cycle:	1	2	3	4	5
IF:	MUL	STALL	STALL	NEXT	

ID:	PREV	MUL	STALL	STALL
EX:	----	MUL	MUL	MUL_wb
MEM:	----	----	----	----
WB:	----	----	----	MUL_res

stall:	0	1	1	0
mul_valid:	0	0	0	1

8. Test Vector Reference

The following tables provide golden input/output pairs for each instruction. These should be embedded in the Verilog testbenches for automated verification. At least 100 test vectors per instruction are recommended (shown here are representative edge cases).

8.1 MUL Test Vectors

```
// a, b, expected MUL (lower 32 bits)
0, 0, 0
1, 1, 1
-1, -1, 1          // 0xFFFFFFFF * 0xFFFFFFFF = lower32 = 1
-1, 1, -1          // 0xFFFFFFFF * 1 = 0xFFFFFFFF
INT_MAX, 2, -2      // 0x7FFFFFFF * 2 = 0xFFFFFFF0 (overflow)
INT_MIN, 1, INT_MIN // 0x80000000 * 1 = 0x80000000
12345, 67890, // verify with Python
```

8.2 MULH / MULHU / MULHSU Test Vectors

```
// MULH (signed x signed, upper 32 bits)
a=-1, b=1: product = 0xFFFFFFFF_FFFFFFFF, upper32 = 0xFFFFFFFF = -1
a=INT_MIN, b=1: product = 0xFFFFFFFF80000000, upper32 = 0xFFFFFFFF = -1
a=INT_MIN, b=INT_MIN: product = 0x4000000000000000, upper32 = 0x40000000

// MULHU (unsigned x unsigned, upper 32 bits)
a=0xFFFFFFFF, b=0xFFFFFFFF: upper32 = 0xFFFFFFF0
a=1, b=0xFFFFFFFF: upper32 = 0

// MULHSU (signed x unsigned, upper 32 bits)
a=-1, b=1: sign_ext(-1) = 0xFFFFFFFFFFFFFFFF * 0x0000000000000001
           = 0xFFFFFFFFFFFFFFFF, upper32 = 0xFFFFFFFF = -1
```

8.3 DIV / DIVU Test Vectors

```
// DIV (signed)
10, 3, 3, 1          // 10 = 3*3 + 1
-10, 3, -3, -1        // -10 = 3*(-3) + (-1)
10, -3, -3, 1          // 10 = (-3)*(-3) + 1
-10, -3, 3, -1         // -10 = (-3)*3 + (-1)
7, 7, 1, 0
0, 5, 0, 0
5, 0, -1, 5           // DIVIDE BY ZERO
INT_MIN, -1, INT_MIN, 0 // OVERFLOW
```

```
// DIVU (unsigned)
10, 3, 3, 1
0, 5, 0, 0
5, 0, 0xFFFFFFFF, 5 // DIVIDE BY ZERO (unsigned returns 2^32-1)
0xFFFFFFFF, 2, 0x7FFFFFFF, 1
```

8.4 MAU Custom Instruction Test Vectors

```
// ABS
0, 0
1, 1
-1, 1 // 0xFFFFFFFF -> 1
INT_MIN, INT_MIN // 0x80000000 -> 0x80000000 (overflow, expected)
12345, 12345
-32768, 32768

// MAX
5, 10, 10
-5, 3, 3 // 3 > -5 (signed comparison)
INT_MIN, INT_MAX, INT_MAX
-1, 0, 0 // 0 > -1 in signed

// MIN
5, 10, 5
-5, 3, -5
INT_MIN, INT_MAX, INT_MIN

// SQRT
0, 0 | 1, 1 | 4, 2 | 100, 10 | 10000, 100
0xFFFFFFFF, 65535

// LOG2
0, 0 | 1, 0 | 2, 1 | 4, 2 | 8, 3 | 255, 7
256, 8 | 1024, 10 | 0x80000000, 31
```

9. Common Bugs and Debugging Checklist

Bug Category	Symptom	Fix
Signed vs Unsigned MUL	MULH gives wrong upper bits for negative numbers.	Ensure proper sign extension: signed operands are sign-extended to 64 bits before multiplication. Use \$signed() in Verilog.
DIV overflow	INT_MIN / -1 crashes or gives wrong result.	Add explicit check: if (dividend == INT_MIN && divisor == -1), return INT_MIN quotient, 0 remainder.
DIV by zero	Hangs in infinite loop or gives X state.	Add pre-check: if (divisor == 0), immediately assert done with quotient = -1 (signed) or 2^32-1 (unsigned).
Pipeline not stalling	Next instruction overwrites MUL/DIV result.	Verify stall signal from execute reaches IF/ID register. Check that PC is frozen during multi-cycle ops.

Write-back timing	Result written to wrong register or not written at all.	Ensure wb_en is asserted only when valid/done signal from the functional unit is high, and rd is correctly passed through pipeline registers.
Control signal missing	Instruction executes as NOP or wrong operation.	Check opcode.vh: verify all funct7/funct3 combinations are decoded. Add new control signals to IF_ID pipeline register.
SQRT off-by-one	$\text{sqrt}(4) = 1$ instead of 2, or $\text{sqrt}(15) = 4$ instead of 3.	Verify FSM iteration count (should be exactly 16 for 32-bit input). Check remainder comparison logic in the last iteration.
MAX/MIN sign bug	MAX(5, -3) returns -3 instead of 5.	Use 33-bit subtraction to detect signed overflow. Standard 32-bit subtraction gives wrong sign when overflow occurs.
Sim vs FPGA mismatch	Works in simulation, fails on FPGA.	Check for uninitialized signals (use rst). Add default case in all case statements. Use ILA to probe internal signals.

10. Python Golden Output Generator

Use the following Python script to generate golden I/O test vectors for your Verilog testbench. The script produces Verilog-compatible test vector format that can be copy-pasted directly into testbenches:

```
import random

def to_u32(v):
    return v & 0xFFFFFFFF

def to_s32(v):
    v = v & 0xFFFFFFFF
    return v - 0x100000000 if v >= 0x80000000 else v

def mul_full(a, b, signed=False):
    if signed:
        a, b = to_s32(a), to_s32(b)
    else:
        a, b = to_u32(a), to_u32(b)
    product = a * b
    if signed:
        product = product & 0xFFFFFFFFFFFFFFFF
    return product

# Example: generate 100 random MUL test vectors
for _ in range(100):
    a = random.randint(0, 0xFFFFFFFF)
    b = random.randint(0, 0xFFFFFFFF)
    result = mul_full(a, b) & 0xFFFFFFFF
    print(f"32'h{a:08x}, 32'h{b:08x}, 32'h{result:08x}")
```